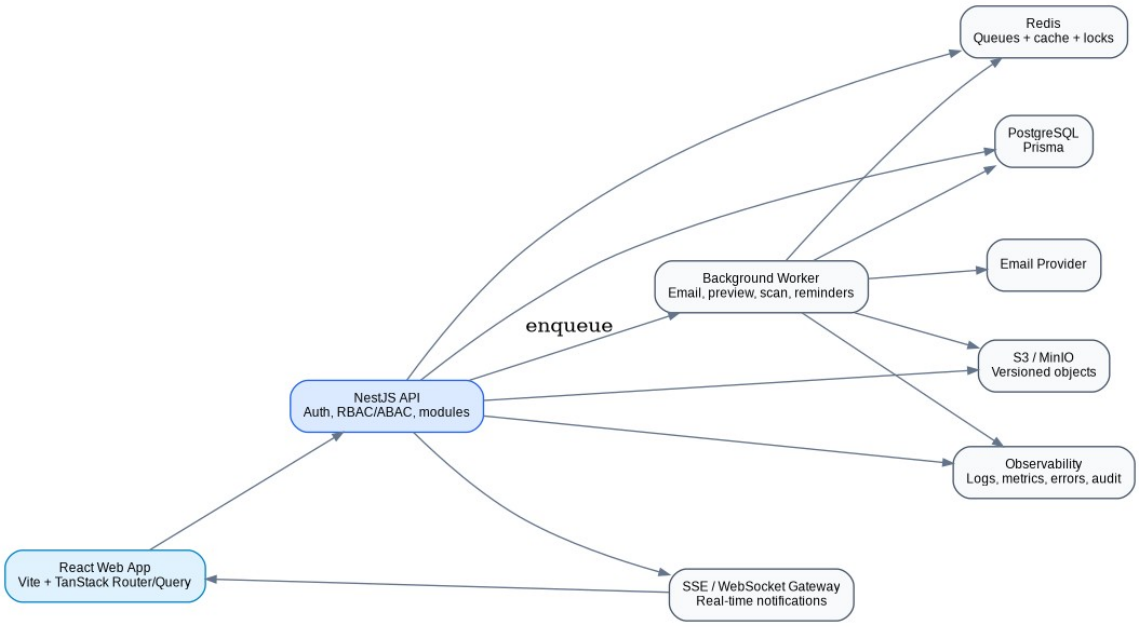
PROJECT PORTAL WORKFLOW MANAGEMENT

Codebase Analysis and Production Rebuild Specification

Version 1.0 | 14 July 2026



Document purpose
A code-evidenced description of the current frontend prototype and an implementation-ready specification for rebuilding it as a secure, maintainable production system. Confirmed prototype behavior and recommended production design are intentionally separated.

Source snapshot: Project-Portal-workFlow-Management(6).zip

Document Control

Field	Value
Document owner	Project Portal product and engineering stakeholders
Document type	Functional specification, technical architecture, gap analysis, and production rebuild plan
Source snapshot	Project-Portal-workFlow-Management(6).zip
Analysis date	14 July 2026
Source files reviewed	80 non-Git, non-node_modules, non-build files; 66 under src; 46 reusable UI primitives
Files that could not be reviewed	None
Current implementation	React 19 + TanStack Start/Router + Vite + Tailwind/Radix; frontend-only state persisted in localStorage
Production status	Working prototype; not production-ready

Evidence rule
Current-state claims cite source paths, line ranges, component names, functions, types, or constants. Proposed architecture, API, data model, and controls are explicitly labeled as recommendations.

Table of Contents

Right-click and update field to refresh the Table of Contents.

The field-based table of contents can be refreshed in Microsoft Word using Update Field. The numbered section list below remains readable in viewers that do not update Word fields.

1. Executive Summary
2. Product Purpose
3. System Scope
4. User Roles and Responsibilities
5. Functional Feature Documentation
6. Page and Screen Documentation
7. Modal and Dialog Documentation
8. Data Model Documentation
9. State Management Documentation
10. Current Frontend Architecture
11. Recommended Production Architecture
12. API Requirements
13. Authorization Model
14. Workflow Engine Design
15. Document Storage Design
16. Notification Design
17. Audit and Activity Logging
18. Validation and Business Rules
19. Error, Warning, and Empty-State Behavior
20. Security Requirements
21. Performance and Scalability
22. Testing Strategy

23. Deployment and DevOps
24. Prototype Gap Analysis
25. Technical Debt and Code Quality Review
26. Recommended Monorepo Structure
27. Development Roadmap
28. Acceptance Criteria
29. Glossary
30. Open Questions
Appendices

Source Review Summary

Category	Reviewed content	Evidence/examples
Application shell and routing	Root shell, error/not-found handling, route tree, header, sidebar, responsive navigation, embedded module dialogs.	`src/router.tsx`, `src/routes/__root.tsx`, `src/routeTree.gen.ts`, `src/components/AppSidebar.tsx`
Business state and workflow	All entities, seed data, localStorage persistence, attachment naming/versioning, bid outcome, 30+ work-request mutations.	`src/lib/portal-data.tsx` - `PortalDataProvider`, `WorkRequestStatus`, context actions
Core feature routes	Dashboard, projects/bids, work requests, documents, notifications, companies/divisions, teams/members, settings.	`src/routes/*.tsx`
Shared presentation	Page headers, status badges, stat cards, workflow timeline, 46 ShadCN/Radix primitives.	`src/components/*`, `src/components/ui/*`
Build and configuration	Vite/TanStack/Cloudflare configuration, dependencies, TypeScript, ESLint, Prettier, Tailwind styles.	`package.json`, `vite.config.ts`, `wrangler.jsonc`, `eslint.config.js`, `src/styles.css`
Existing project notes	Prototype intent, actor model, route flow, and simulated production functions.	`IMPLEMENTATION_NOTE.md`, `README.md`

Validation	Result
Production build	Successful for client and SSR environments.
Largest client JavaScript chunk	780.28 KB minified / 225.46 KB gzip; Vite emitted the >500 KB chunk warning.
Largest server chunks	router: 1,103.00 KB; worker entry: 732.78 KB.
Lint	Failed with 741 findings: 698 errors and 43 warnings; formatting dominates, with additional React hook/Fast Refresh warnings.
Code changes during documentation	None.

1. Executive Summary

Project Portal Workflow Management is a role-aware engineering document and project coordination prototype. It connects bid intake, active projects, work requests, PMO/ECM preparation, TMS drawing/checking/approval, CCR/Marketing delivery, client decisions, document revision, and final HML registry listing in one browser application.

The prototype demonstrates the intended business sequence and most user-facing interactions, but it is not a secure multi-user system. Authentication is simulated by a header role selector, authorization is implemented only as UI visibility logic, all records and file data URLs are serialized into browser localStorage, and notifications are derived from the current in-memory workflow state. These constraints make it suitable for stakeholder validation but unsafe for production operations.

The recommended production target is a modular monolith in an Nx and pnpm monorepo: a React web application, a NestJS API, PostgreSQL/Prisma persistence, S3-compatible object storage, Redis-backed queues and locks, a background worker, centralized RBAC/ABAC authorization, durable audit logs, and real-time plus email notifications. Microfrontends are not recommended at this stage because the domains share one workflow, one identity boundary, and frequent cross-module transactions.

Business outcome	Prototype evidence	Production requirement
Controlled project lifecycle	Bids/projects and related work requests move through coded statuses and history entries.	Server-side state machine, transactions, optimistic locking, configurable SLAs.
Traceable technical review	Revision history records actor label, action, from/to, note, and timestamp.	Immutable audit event store plus searchable business activity feed.
Structured document handling	Generated root/category/date/revision folders, generated names, version numbers, preview/download.	Object storage, presigned URLs, MIME validation, virus scan, preview pipeline, retention.

Business outcome	Prototype evidence	Production requirement
Role-focused work queues	Sidebar visibility, work-request filters, action availability, and notifications are actor-aware.	Backend resource authorization with least privilege and policy tests.
Client acceptance loop	Client can provide info, accept, reject, or request revisions; accepted bids can become projects.	External client identity, tenant isolation, e-sign/acknowledgment, durable delivery evidence.

2. Product Purpose

2.1 Primary business purpose

The portal is intended to replace fragmented email, chat, spreadsheets, and shared-drive coordination with a single controlled workspace. CCR/Marketing creates a bid or project shell, receives client information, creates one or more work requests, and routes each request to an engineering or operations division. The work package is prepared, checked, approved, delivered to the client, revised when required, and finally listed as an accepted HML document.

2.2 Core relationships

Concept	Relationship
Client	Owns or receives one or more bids/projects. Client-mode visibility is filtered by `clientId`.
Bid / Project	Parent workspace containing primary intake files and all workflow files. A won bid can be converted to a project.
Work Request	A scoped unit of engineering/operations work under exactly one bid or project. It carries status, assignments, info requests, file selections, and history.
Document / Attachment	Stored inside the parent project `initialDocuments` array; tagged as PRIMARY or WORKFLOW and optionally linked to a work request.
Revision	Represented both by numbered attachment versions and structured revision codes; workflow revision is represented by status/history.
Registry Document	A final logical listing created after acceptance or a legacy final-list action.
Notification	A derived per-actor work item based on the latest workflow history entry and action availability.

2.3 End-to-end lifecycle

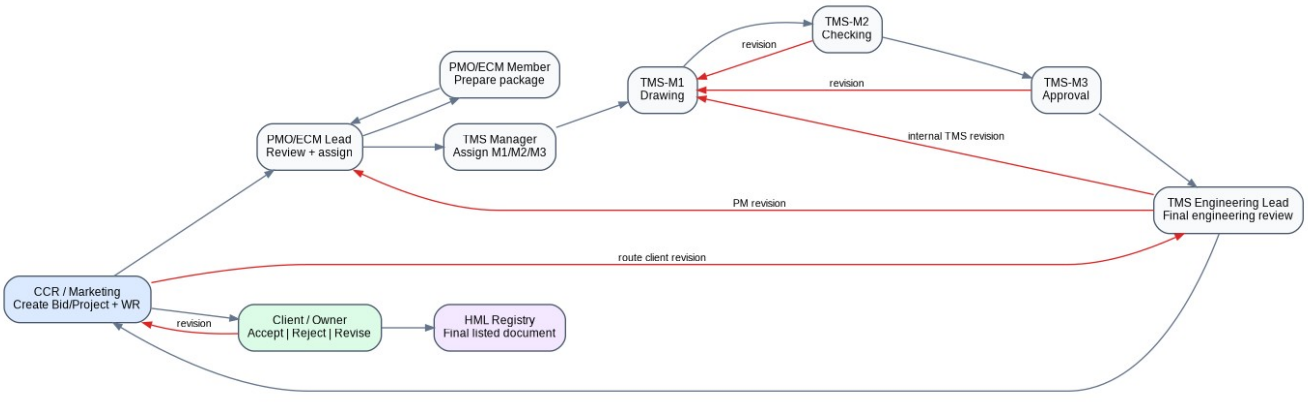


Figure 1. Confirmed prototype workflow and primary revision loops.

1. CCR/Marketing creates a bid or direct project and uploads initial client documents.
2. CCR/Marketing creates a work request under an existing bid or active project and assigns PMO or ECM.

3. The PM/engineering lead requests missing information or assigns a division member.
4. The division member prepares the package, asks questions if needed, and submits it to the lead.
5. The lead requests rework or forwards the approved package to TMS.
6. The TMS manager assigns M1 drawing, M2 checking, and M3 approval members.
7. M1 submits a drawing; M2 and M3 approve or return it to M1.
8. The TMS engineering lead requests PM/TMS revision or submits the final package to CCR/Marketing.
9. CCR/Marketing requests engineering revision or sends the final package to the client.
10. The client accepts, rejects, or requests revision. Acceptance creates/finalizes a registry listing; accepted bids move to client-pending before win/loss assignment.

3. System Scope

3.1 Confirmed prototype scope

- Actor/role simulation and role-aware UI.
- Dashboard KPIs, quick actions, workspaces, critical submissions, and activity.
- Bid/project creation, filtering, document upload, bid decision, archive, and archive review.
- Work-request creation, assignment, action dialogs, information requests, stage transitions, history, file selection, and revision handling.
- Document explorer with generated hierarchy, search, list/grid modes, preview, download, and workflow upload.
- Derived notification inbox with unread/read persistence and deep navigation.
- Companies, divisions, teams, members, and configurable naming/dropdown data.

3.2 Recommended production scope

- Real employee/client authentication, invitation, password reset, MFA option, session control, and account lifecycle.
- Server-side authorization, tenant/client isolation, division/assignment scoping, and auditable policy decisions.
- PostgreSQL persistence, transactional workflow transitions, immutable audit events, and concurrency protection.
- S3/MinIO storage, malware scanning, preview generation, version retention, and signed downloads.
- Durable in-app/email/real-time notifications, reminders, retries, user preferences, and escalation policies.
- Search, reporting, exports, operational dashboards, observability, backups, and disaster recovery.

3.3 Out of scope for the first production release

- Microfrontend runtime federation.
- Native mobile applications.
- Full CAD/BIM editing inside the browser.
- Public anonymous file sharing.
- Automatic engineering calculation or design approval.
- Complex billing or ERP accounting.

3.4 Future enhancements

- Microsoft 365/Graph integration.
- Digital approval signatures and delivery receipts.
- Workflow SLA timers and automated escalation.
- Document comparison and markup.
- Configurable workflow templates by work-request type.
- Cross-project portfolio analytics and capacity planning.

4. User Roles and Responsibilities

The code defines ten actor-role identifiers. PMO and ECM are not separate role enums; they are differentiated by the member division while sharing `division_lead` and `division_member`. CCR/Marketing behavior is represented by `ccr_coordinator`.

Role	Code	Responsibility	Visibility	Code evidence
System Admin	`system_admin`	Global prototype supervisor; can manage organization/settings and	All projects, bids, work requests, documents, settings.	`ActorRole`; supervisor checks in `work-requests.tsx` and dashboard.

Role	Code	Responsibility	Visibility	Code evidence
		perform most workflow actions.		
Prime Consultant	`prime_consultant`	Global business/workflow supervisor with broad access equivalent to admin for most prototype actions.	All workspaces plus system settings.	`actorCanManageProjects`; `actorCanCreateWorkRequests`; `canViewSystem`.
CCR Coordinator / Marketing	`ccr_coordinator`	Creates bids/projects/work requests, resolves information, delivers final packages, routes client revisions, manages bid outcome/review.	All portal workspaces.	`PortalDataProvider` marketing actions; `ProjectsPage`; action permission switch.
PMO/ECM Lead	`division_lead`	Manages assigned/origin-division work, assigns member, resolves questions, reviews package, forwards to TMS.	Accessible work requests in matching assigned/origin division.	Member division plus `isDivisionLeadFor()` and `canPerformAction()`.
PMO/ECM Member	`division_member`	Prepares assigned work package, asks for information, responds to rework, submits to lead.	Assigned work requests only, plus info-request participation.	`assignedMemberId` filtering and action checks.
TMS Manager / Engineering Lead	`tms_manager`	Assigns TMS chain, answers TMS questions, requests PM/TMS revision, releases final engineering package.	TMS-assigned and relevant division work requests.	`assignTmsChain`, `engineeringSubmitToMarketing`, TMS action checks.
TMS-M1 Drawing	`tms_drawing`	Produces/revises drawing and submits to checking.	Work requests where actor matches `tmsAssignments.drawingId`.	`submitDrawing`; assigned TMS visibility.
TMS-M2 Checking	`tms_checking`	Approves drawing or requests revision.	Work requests where actor matches `checkingId`.	`reviewChecking`.
TMS-M3 Approval	`tms_approval`	Final technical approval or revision request.	Work requests where actor matches `approvalId`.	`reviewApproval`.
Client / Owner	`client_owner`	Views own client workspaces, provides requested information, accepts/rejects/requests revision.	Projects where `project.clientId === actor.clientId`.	Sidebar/project filters and client action checks.

4.1 Role-permission matrix

Role	Manage org	Create bid/project/WR	Data scope	Workflow capability	Documents	Settings
System Admin	Yes	Yes	All	All prototype actions	Yes	Yes
Prime Consultant	Yes	Yes	All	All prototype actions	Yes	Yes
CCR Coordinator	Yes	Yes	All	Marketing/client routing, bid outcome, final listing	Upload	No
Division Lead	No	No	Assigned/origin division	Assign member, info, PM review, forward	Upload	No
Division Member	No	No	Assigned only	Info, prepare, submit, rework	Upload	No
TMS Manager	No	No	Assigned/TMS relevant	Assign TMS, respond, revise, final release	Upload	No
TMS-M1	No	No	Assigned M1	Submit drawing/request info	Upload	No
TMS-M2	No	No	Assigned M2	Approve/return/ request info	Upload	No
TMS-M3	No	No	Assigned M3	Approve/return/ request info	Upload	No
Client	No	No	Own client	Provide info, accept/reject/revise	Own workspace upload	No

Security warning
These checks only hide or show frontend elements and choose which local records to display. They are not security controls.

Production authorization must be repeated and enforced in the API and storage layer.

5. Functional Feature Documentation

Module	Confirmed behavior	Evidence
Authentication / authorization simulation	Header actor selector changes `currentActorId`; sidebar, filters, actions, and notifications react to the selected actor.	`src/routes/__root.tsx`; `PortalDataProvider.setCurrentActorId`; `AppSidebar`.
Dashboard	KPIs, quick actions, project/bid tables, workspace folders, critical submissions, routing activity, workflow detail, history, embedded modules.	`src/routes/index.tsx` (4,108 lines).
Bid and project management	Create, upload, filter, view, progress, archive, review, win/lose conversion, no-work-request warning.	`src/routes/projects.tsx`; `addProject`, `decideBidOutcome`, `requestArchivedBidReview`.
Work requests	Create under parent, assign division/member/TMS chain, actions, notes, file uploads, history, preview/download, final selection.	`src/routes/work-requests.tsx`; context mutation methods.
Information requests	Targeted request, nested stack, response backtracking, plus legacy marketing/client escalation statuses.	`PendingInfoRequest`; `requestWorkflowInfo`; `respondWorkflowInfo`.
Documents	Explorer tree, generated folders, search, grid/list, upload, image/PDF/text preview, metadata fallback, download.	`src/routes/documents.tsx`; attachment naming helpers.
Versioning and revisions	Same stable key increments version; revision upload receives BID letters or project numeric codes.	`getNextAttachmentVersion`; `getNextRevisionCode`; `createStructuredNamingMeta`.
Notifications	Per-actor derived inbox, action labels, unread/read state, persistent capped inbox, deep navigation.	`src/routes/notifications.tsx`; per-actor localStorage keys.
Organization	View/add companies and divisions.	`src/routes/companies.tsx`; `addCompany`; `addDivision`.
Teams and members	Search, team creation, lead selection, member validation/creation, unassigned list, role options by division.	`src/routes/teams.tsx`; `addTeam`; `addMember`.
Settings	Portal name, POL/POD, cargo/vessel codes, document catalogs, work-request types, project info, attachment categories, resets.	`src/routes/settings.tsx`; `PortalSettings`; `updateSettings`.

5.1 Bid/project lifecycle rules

- New BID: code `BID-26xxx`, status `BIDDING`, origin division CCR, credentials marked sent (simulated).
- New direct PROJECT: code `PRJ-26xxx`, status `ACTIVE`.
- Client acceptance of a bid work request moves the bid to `DRAFT` with progress 100, described in UI as client-pending.
- Bid outcome can be decided only when at least one related work request is `HML_LISTED`.
- WIN converts the same record from BID to PROJECT, changes code prefix to PRJ, sets ACTIVE, and rewrites related work-request `parentType`.
- LOSE sets ARCHIVED and progress 100. Archive review sets BIDDING and reopens a selected work request at final client review.

5.2 Complete workflow transition catalog

Current state	Action	Allowed actor	Result	Conditions/side effects
Any active status	Request Info	Role-dependent target list	Status unchanged; set `pendingInfoRequest`; previous pending request pushed to stack	Target must exist and cannot be self.
Any with pending info	Respond to Info	Target or supervisor	Status unchanged; history added; previous stacked request restored	Pending target match.
DIVISION_NOTIFIED / LEADER_ASSIGNED /	Request Marketing Info	Lead/supervisor	INFO_REQUESTED_FROM_MARKETING	Lead/origin lead or supervisor.

Current state	Action	Allowed actor	Result	Conditions/side effects
MARKETING_INFO_PROVIDED / revision states				
INFO_REQUESTED_FROM_MARKETING or CLIENT_INFO_PROVIDED	Return Updated Info to PM	CCR/supervisor	MARKETING_INFO_PROVIDED	Marketing review complete.
INFO_REQUESTED_FROM_MARKETING	Escalate to Client	CCR/supervisor	INFO_REQUESTED_FROM_CLIENT	Missing client input.
INFO_REQUESTED_FROM_CLIENT	Provide Requested Info	Client/supervisor	CLIENT_INFO_PROVIDED	Client actor or supervisor.
DIVISION_NOTIFIED	Assign Leader	Supervisor only in UI	LEADER_ASSIGNED	Leader ID required.
DIVISION_NOTIFIED / LEADER_ASSIGNED / returned-info/revision states	Assign PM Member	Lead/supervisor	MEMBER_REVIEW	Member ID; leader auto-filled from current actor when absent.
MEMBER_REVIEW / PM_REWORK_REQUESTED / PM_LEAD_RESPONDED	Member Requests Info	Assigned member/supervisor	MEMBER_INFO_REQUESTED	Assigned member.
MEMBER_INFO_REQUESTED	Lead Responds	Lead/supervisor	PM_LEAD_RESPONDED	Assigned/origin lead.
MEMBER_REVIEW / PM_REWORK_REQUESTED / PM_LEAD_RESPONDED	Submit Work to PM Lead	Assigned member/supervisor	PM_MEMBER_SUBMITTED	Optional file upload.
PM_MEMBER_SUBMITTED	Request PM Rework	Lead/supervisor	PM_REWORK_REQUESTED	Revision comment; optional files.
PM_MEMBER_SUBMITTED	Forward to TMS	Lead/supervisor	FORWARDED_TO_TMS	Lead approval.
FORWARDED_TO_TMS or routed CCR revision	Assign TMS Chain	TMS manager/supervisor	TMS_ASSIGNED	M1, M2, M3 required.
TMS_ASSIGNED / DRAWING_IN_PROGRESS	Submit Drawing	Assigned M1/supervisor	CHECKING_REVIEW	Document name and optional file.
CHECKING_REVIEW	M2 Approve	Assigned M2/supervisor	APPROVAL_REVIEW	Approve branch.
CHECKING_REVIEW	M2 Request Revision	Assigned M2/supervisor	DRAWING_IN_PROGRESS	Return to M1 with comment.
APPROVAL_REVIEW	M3 Approve	Assigned M3/supervisor	ENGINEERING_LEAD_REVIEW	Approve branch.
APPROVAL_REVIEW	M3 Request Revision	Assigned M3/supervisor	DRAWING_IN_PROGRESS	Return to M1.
TMS work stages	TMS Member Requests Info	Assigned M1/M2/M3	TMS_MEMBER_INFO_REQUESTED	Previous status/requester stored.
TMS_MEMBER_INFO_REQUESTED	TMS Lead Responds	TMS manager/supervisor	Restores previous TMS status	Stored status/requester cleared.
FORWARDED_TO_TMS / ENGINEERING_LEAD_REVIEW	Request PM Revision	TMS manager/supervisor	ENGINEERING_REVISION_REQUESTED	Return to assigned division lead.
ENGINEERING_LEAD_REVIEW	Request Internal TMS Revision	TMS manager/supervisor	DRAWING_IN_PROGRESS	Return to assigned M1.
ENGINEERING_LEAD_REVIEW	Submit Final to CCR/Marketing	TMS manager/supervisor	FINAL_SUBMITTED_TO_MARKETING	Select latest engineering package; fallback by workflow stages.
FINAL_SUBMITTED_TO_MARKETING	Submit Final to Client	CCR/supervisor	FINAL_SUBMITTED_TO_CLIENT	Selected/uploaded marketing files become final package.
FINAL_SUBMITTED_TO_MARKETING	Request Engineering Revision	CCR/supervisor	ENGINEERING_LEAD_REVIEW	Return to TMS engineering lead.
FINAL_SUBMITTED_TO_CLIENT / REJECTED	Client Accept	Client/supervisor	HML_LISTED	Create registry row if absent; bid becomes DRAFT/client-pending.
FINAL_SUBMITTED_TO_CLIENT	Client Reject	Client/supervisor	REJECTED	Parent normally unchanged; archived review rejection rearchives bid.
FINAL_SUBMITTED_TO_CLIENT / REJECTED	Client Requests Revision	Client/supervisor	CLIENT_REVISION_REQUESTED	Latest client-revision files recorded.
CLIENT_REVISION_REQUESTED	Route Revision to TMS	CCR/supervisor	ENGINEERING_LEAD_REVIEW	Latest history marks CCR-to-TMS revision; TMS assignment can reopen.
CLIENT_REVISION_REQUESTED	Submit Client Revision Response	CCR/supervisor	FINAL_SUBMITTED_TO_CLIENT	Direct updated response path.
RETURNED_TO_DIVISION	Legacy Member Review	Assigned member/supervisor	DIVISION_MEMBER_APPROVED or FORWARDED_TO_TMS	Legacy path remains in code.
DIVISION_MEMBER_APPROVED	Legacy Manager Approve	Origin lead/supervisor	DIVISION_MANAGER_APPROVED	Legacy path.

Current state	Action	Allowed actor	Result	Conditions/side effects
DIVISION_MANAGER_APPROVED	Legacy Forward to CCR	Origin lead/supervisor	FORWARDED_TO_CCR	Legacy path.
FORWARDED_TO_CCR	Legacy Final Listing	CCR/supervisor	HML_LISTED	Creates registry document.

Confirmed inconsistency
`marketingSendClientRevisionToPm()` exists and changes the state to `ENGINEERING_REVISION_REQUESTED`, but `canPerformAction(..., "MARKETING_ROUTE_CLIENT_REVISION_PM")` always returns false. The function is therefore effectively unreachable through the current action UI and must be resolved during requirements finalization.

5.3 Document naming and folder rules

```
BID generated filename:
{biddingNumber} {projectCode}-{cargoCode}-{vesselCode}-{shipmentNumber}-{documentCode}[-{revisionCode}]
{documentName}.{ext}

PROJECT generated filename:
{projectCode}-{cargoCode}-{vesselCode}-{shipmentNumber}-{documentCode}[-{revisionCode}] {documentName}.{ext}

Folder path:
{rootFolderName} / {documentCode} {documentName} / {DD-MM-YYYY [Rev X]} / {generatedFileName}

Root folder fallback:
{BID number or project code} - {client} - {project name} - {cargo name}
```

Rule	Confirmed behavior
Code normalization	Uppercase alphanumeric only; empty values fall back to `XXX`.
Shipment number	Digits only, maximum two digits, left-padded to two; default `01`.
Project naming code	First letter of project name + POL + POD.
Revision code	BID: A, B, ... Z, AA; PROJECT: 01, 02, ...
Version number	Incremented for matching stable attachment key in the scoped primary/work-request files.
Revision vs version	A revision upload can also create a new file version; the fields represent different concepts.
File storage	Original filename is preserved as metadata, but generated filename is used for display/path.

6. Page and Screen Documentation

Route/screen	Purpose	Authorization	Main functions	Navigation/state
/	Dashboard and contextual bid/project/work-request workspace	All actors; data scoped by role	KPIs, quick actions, project/bid/workspace lists, critical submissions, activity, workflow detail	Search parameters: `bidId`, `projectId`, `requestId`; numerous embedded dialogs.
/projects	Bid and project management	All actors can view scoped data; admin/prime/CCR can create/manage	Search, filters, cards/table, create, upload, details, bid win/lose/archive/review	Client filter; warning when cycle opened with no work request.
/work-requests	Work-request queue and workflow execution	All actors with role/assignment scope	Create, group, expand, detail, action, upload, history, preview, final listing	Search: `requestId`, `view`; modal-only embedding supported.
/documents	Structured document explorer	Actors with accessible projects	Folder tree, breadcrumb, search, list/grid, upload, preview, download	Search: `projectId`; upload requires a work request.
/notifications	Notification center	Current actor	Derived queue, unread	Also embedded in header

Route/screen	Purpose	Authorization	Main functions	Navigation/state
			indicator, mark read/all, deep navigation	dropdown.
/companies	Companies and divisions	View available; manage admin/prime	Cards, add company, add division	No edit/delete in prototype.
/teams	Teams and members	View available; manage admin/prime	Search, team cards, unassigned members, create team/member	Email/duplicate/team validation.
/settings	Portal configuration	Sidebar restricted to admin/prime	General and editable list/code groups, add/remove/reset	Updates global state/localStorage immediately.
Root shell	Application chrome	All actors	Desktop/mobile sidebar, actor selector, notification dropdown, module dialogs, toaster	No real login/logout.

6.1 Screen-state observations

- Loading states are mostly absent because all data is local and synchronous.
- Error handling is primarily toast messages; there is a root error component and 404 page.
- Empty states are implemented for document folders, work queues, notifications, and sidebar lists.
- Several modules are usable both as routes and as dialogs embedded inside the dashboard/root shell.
- The dashboard duplicates major work-request action-availability logic rather than calling one shared policy function.

7. Modal and Dialog Documentation

Dialog	Trigger	Authorized role	Inputs/content	Submit/behavior
Root Module Dialog	Header/module action	Varies by module	Hosts Projects, Work Requests, Documents, Organization, Teams, Settings	Switches embedded module state; closes on dialog close.
Dashboard Modal	Dashboard quick action/KPI/workspace	Current actor	Projects, work requests, documents, lists, cycle, activity, history	Modal title derived from `DashboardModalState`.
Create Bid / Project	Projects create button / dashboard action	Admin, Prime, CCR	Type, name, client, source, bid naming fields, one or more initial documents	Validates required fields and at least one document; creates credentials simulation.
Upload Workflow Document	Project row/detail upload	Accessible actor/client owner of project	Document option, name/category/stage, file/text/note	Adds workflow attachment.
Project Details	View action	Accessible actor	Metadata and all documents	Preview/download and upload.
Create Work Request	Quick action / parent workspace	Admin, Prime, CCR	Parent, title, type/category, priority, attachment category, division, notes, files	Must be attached to valid bid/project; can lock parent type/id.
Work Request Details	View details	Accessible actor	Metadata, assignments, workflow, files, feedback	Read-only details.
Workflow Action	Action button/notification	Role and status dependent	Dynamic selector, note/comment, document code/files, assignments/decisions	Executes one transition; resets/close after success.
Upload Workflow / Collaboration File	Upload button	Accessible actor	Document code, multiple files, note	Same file creates new version; revision states set revision flag.
History	History button	Accessible actor	Transfer and revision history	Timeline view.
File Preview	File action	Accessible actor	Image/PDF/text preview or unsupported message	Download available.
Final HML Document Listing	Legacy final-list action	CCR/supervisor	Name and category	Sets HML_LISTED and creates registry row.
Upload File - Documents	Explorer upload button	Accessible project; client must own	Work request, document code, files, note	Work request is mandatory.
Add Company	Organization action	Admin/Prime	Name, abbreviation, type	Name/abbreviation required.
Add Division	Company action	Admin/Prime	Name, abbreviation, type	Company context plus required name/abbr.
Create Team	Teams action	Admin/Prime	Name, company, division,	Duplicate team check within

Dialog	Trigger	Authorized role	Inputs/content	Submit/behavior
			optional lead	division.
Add Member	Teams action	Admin/Prime	Name, email, company, division, team, role title	Email format and duplicate validation.
Add Settings Item	Settings group action	Admin/Prime via route visibility	Text item or name+code	Duplicate validation; code normalized.

Modal handoff rule

The dashboard/root module containers replace their current modal state when opening a nested module. The work-request create flow also supports `embeddedModalOnly` and `onEmbeddedModalClose`, which should be preserved as an explicit navigation/modal orchestration service in the rebuild.

8. Data Model Documentation

The current model is a TypeScript object graph, not a database schema. The production model below normalizes identities, assignments, attachments, versions, workflow events, notification deliveries, and audit records.

Entity	Confirmed fields	Relationships	Validation/status
Company	id, name, abbr, type	Has divisions, members, clients	No update/delete validation
Division	id, companyId, name, abbr, type	Belongs to company; has teams/members	Company existence only implied by UI.
Team	id, name, companyId, divisionId, leadMemberId?, memberIds[]	Belongs to company/division; members	Duplicate name checked in UI.
Member	id, name, email, companyId, divisionId, teamId?, roleTitle, active	May map to actor	Email/required/duplicate checked in Teams page.
Client	id, name, email, companyId	Owns projects/bids	Seeded only; no client management route.
Actor	id, label, role, memberId?, clientId?	Current simulated identity	Exactly one currentActorId.
ProjectItem	id, code, name, type, clientId, clientEmail, originDivisionId, sourceChannel, status, progressOverride?, bidDetails?, initialDocuments[], credentialsSent, createdAt	Parent of work requests and registry documents	Status enum DRAFT/BIDDING/ACTIVE/COMPLETED/ARCHIVED.
BidNamingDetails	biddingNumber, projectName, POL/POD, project/cargo/vessel codes, shipment, rootFolderName	Embedded in project	UI validates bid fields.
WorkRequest	id, code, parentType/id, title, category, priority, attachmentCategory, notes, assigned/origin division, status, assignments, final/revision file IDs, info request, history	Belongs to project; has assignments/history/files	Core workflow state.
PendingInfoRequest	id, requester identity/role, target role/member/client, statusAtRequest, note, requestedAt	Embedded current request plus stack	Supports nested routing.
AttachmentRef	identity, display/naming fields, uploader/time, source/file group, text/file data, type/size, version, workRequestId, revision/folder metadata	Stored inside ProjectItem.initialDocuments	Data URL stored inline.
HistoryEntry	id, at, by, action, from?, to?, note?	Embedded in work request	Newest first; mutable browser state.
RegistryDocument	id, projectId, workRequestId, name, category, divisionId, listedAt/by	Final logical document list	Created on acceptance/listing.
PortalSettings	portalName, categories, types, POL/POD, code catalogs, document catalogs	Global configuration	No versioning/audit.
NotificationItem	derived id/kind/resource/title/status/message/time/actor/target view	Derived from project/work request and history	Stored separately per actor, capped at 150.

8.1 Recommended production ERD

```
erDiagram
  ORGANIZATION ||--o{ DIVISION : contains
```

```
DIVISION ||--o{ TEAM : contains
USER ||--o{ TEAM_MEMBERSHIP : joins
TEAM ||--o{ TEAM_MEMBERSHIP : has
CLIENT ||--o{ PROJECT : owns
PROJECT ||--o{ WORK_REQUEST : contains
WORK_REQUEST ||--o{ WORKFLOW_EVENT : records
WORK_REQUEST ||--o{ WORK_ASSIGNMENT : assigns
WORK_REQUEST ||--o{ INFORMATION_REQUEST : has
PROJECT ||--o{ DOCUMENT : stores
DOCUMENT ||--o{ DOCUMENT_VERSION : versions
DOCUMENT_VERSION ||--o{ FILE_OBJECT : points_to
WORK_REQUEST ||--o{ WORK_REQUEST_DOCUMENT : links
USER ||--o{ NOTIFICATION : receives
USER ||--o{ AUDIT_EVENT : performs
PROJECT ||--o{ REGISTRY_ENTRY : produces
```

8.2 Production entity decisions

- Use a single `users` table with organization/client relationships; roles and permissions are separate assignments, not hard-coded actor records.
- Store workflow status on `work_requests`, with every transition in immutable `workflow_events`.
- Store document metadata separately from physical `file_objects`; one document can have many versions.
- Represent revision code, file version, and workflow cycle separately.
- Use join tables for work-request document selections and final/client-revision packages.
- Store notification event, delivery channel, recipient, read time, and delivery status separately.

9. State Management Documentation

State category	Current implementation	Risk	Production approach
Global business state	One `PortalState` in `PortalDataProvider`.	Every change can rerender large application areas; mixed responsibilities.	TanStack Query for server state; small auth/UI stores only.
Persistence	Entire state JSON in localStorage key `project-portal-state-v13-naming-structure`.	Quota, corruption, no multi-user sync, no transactions.	API + PostgreSQL; local cache is disposable.
File data	Data URLs embedded in attachments and serialized into state.	Rapid quota exhaustion and memory overhead.	Direct-to-object-storage uploads.
Mutations	Context callback clones state via JSON serialization and mutates clone.	Slow for growth; loses non-JSON types; no concurrency.	Transactional domain services and typed API commands.
Derived data	Repeated `useMemo` filters across sidebar/routes.	Duplicated policy logic and inconsistent behavior.	Shared selectors/client policies backed by server scopes.
Forms	Mostly `useState` and manual validation.	Inconsistent errors and reset behavior.	React Hook Form + Zod shared schemas.
Notifications	Derived live from latest history and merged with local inbox.	No true event delivery, deduplication guarantee, or cross-device read state.	Event/outbox + notification service.

9.1 Recommended frontend state boundaries

- Server state: projects, requests, documents, organization, settings, notifications - TanStack Query.
- Authentication state: current session, user, effective permissions - secure session endpoint plus minimal memory store.
- UI state: open dialogs, selected tabs, table display settings - local component state or small Zustand store.
- Form state: React Hook Form with Zod resolver.
- Workflow state: server is authoritative; UI receives allowed action descriptors and current version token.
- Cached data: query-keyed, invalidated by mutations and real-time events; never used as an authorization source.

10. Current Frontend Architecture

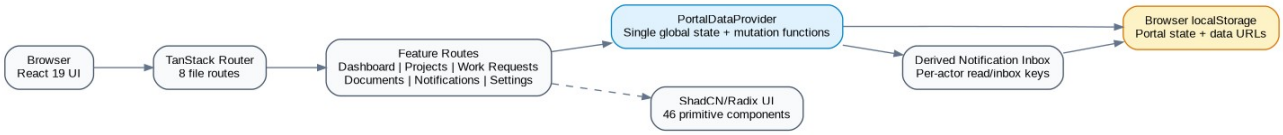


Figure 2. Current prototype architecture.

Area	Current technology/behavior	Code evidence
Framework	React 19.2 with TanStack Start/Router and Vite 7.	`package.json`, `src/router.tsx`.
Routing	Eight file routes plus root shell; route search drives contextual dialogs/workspaces.	`src/routes/*`, `routeTree.gen.ts`.
Styling/UI	Tailwind CSS 4, Radix/ShadCN primitives, Lucide icons, Sonner toasts.	`src/styles.css`, `src/components/ui/*`.
State	React context and hooks; no server API.	`PortalDataProvider`.
Forms	Manual `useState`; React Hook Form wrapper exists but business pages do not use it.	Only `src/components/ui/form.tsx` imports RHF.
Validation	Manual imperative checks and toast messages.	Projects, Work Requests, Teams, Companies, Settings routes.
Deployment target	TanStack Start server build and Cloudflare Wrangler configuration.	`vite.config.ts`, `wrangler.jsonc`.
Error handling	Root error component, not-found page, toast errors.	`src/router.tsx`, `src/routes/__root.tsx`.

10.1 Architectural weaknesses

- `src/routes/work-requests.tsx` is 5,561 lines; `index.tsx` is 4,108; `portal-data.tsx` is 3,087.
- UI, workflow policy, storage, naming, mutation, and notification behavior are tightly coupled.
- Action authorization is duplicated between work-request and dashboard implementations.
- Generic and legacy workflow paths coexist without a clear migration boundary.
- No API abstraction, repository layer, schema version migration, automated tests, or telemetry.

11. Recommended Production Architecture

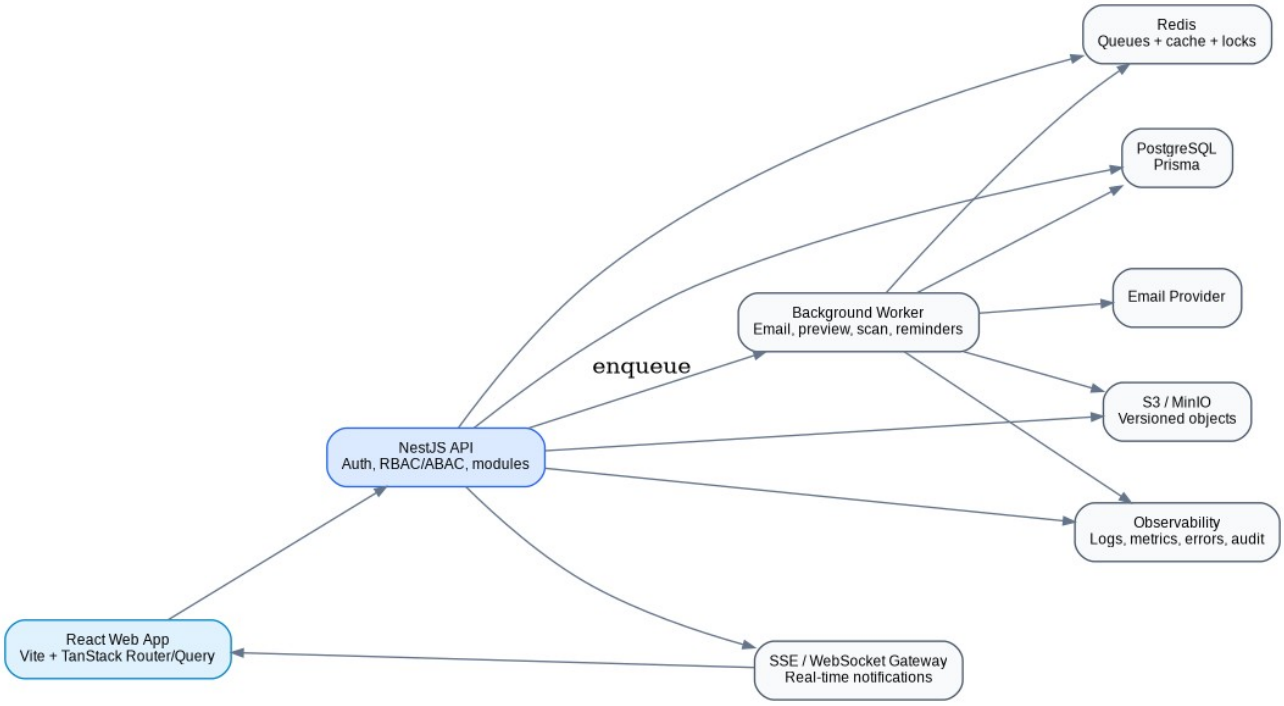


Figure 3. Recommended production architecture.

Use a modular monolith with strong domain package boundaries. This preserves transactional workflow consistency and avoids premature distributed-system complexity. The worker and object store are separate runtime concerns, but business ownership stays inside one API codebase.

Layer	Recommendation	Responsibilities
Web	React + TypeScript + Vite + TanStack Router/Query + RHF/Zod + Tailwind/Radix	Presentation, accessible forms, query cache, optimistic UI, policy-based visibility.
API	NestJS modular monolith	Authentication, authorization, commands/queries, workflow state machine, documents, notifications, reports.
Database	PostgreSQL + Prisma	Transactional source of truth, constraints, indexes, row versions, outbox/audit.
Queue/cache	Redis + BullMQ	Background jobs, distributed locks, rate limits, short-lived cache.
Storage	Amazon S3 or MinIO	Versioned binary objects, presigned upload/download, lifecycle policies.
Worker	Node/Nest worker	Malware scan orchestration, preview conversion, email, reminders, exports.
Realtime	SSE initially; WebSockets only when bidirectional use cases require it	Notification and work-item refresh.
Observability	OpenTelemetry + structured logs + error tracking + metrics	Trace commands/jobs, diagnose failures, measure SLAs.

11.1 Key architecture principles

- API is authoritative for permissions, workflow transitions, numbering, versioning, and final package selection.
- Commands are idempotent where retries are possible and include an expected row/workflow version.
- Every successful business mutation writes an audit event and outbox event in the same database transaction.
- Binary uploads do not pass through the main API process except small metadata/control calls.

- Frontend receives `availableActions` from the API but still displays human-readable permission failures when stale.
- No package may import another domain implementation directly; communicate through exported application services/contracts.

12. API Requirements

Recommended base path: `/api/v1`. All mutating endpoints require authentication, permission/resource policy checks, validation, idempotency where relevant, an audit event, and a correlation ID. List endpoints use cursor or page pagination and server-side filtering.

Module	Method	Endpoint	Purpose	Authorization	Request	Response
Auth	POST	/auth/login	Authenticate employee/client	Public	email, password	Session/user/permissions
Auth	POST	/auth/refresh	Rotate/refresh session	Session	refresh cookie	New access/session state
Auth	POST	/auth/logout	Revoke session	Authenticated	-	204
Auth	POST	/auth/forgot-password	Send reset link	Public	email	202
Auth	POST	/auth/reset-password	Set password from token	Public token	token, password	204
Users	GET	/users	List/search users	users.read	filters/page	User page
Users	POST	/users	Create/invite user	users.create	profile, org, roles	User
Users	PATCH	/users/:id	Update/activate user	users.update	patch	User
Roles	GET	/roles	List roles/permissions	roles.read	-	Role list
Roles	PUT	/users/:id/roles	Assign roles/scopes	roles.assign	role assignments	Assignments
Organization	GET	/organizations	List companies/divisions	org.read	filters	Hierarchy
Organization	POST	/organizations	Create company	org.manage	name, abbr, type	Organization
Organization	POST	/organizations/:id/divisions	Create division	org.manage	name, abbr, type	Division
Teams	GET	/teams	List/search teams	teams.read	filters/page	Team page
Teams	POST	/teams	Create team	teams.manage	name, division, lead	Team
Teams	POST	/teams/:id/members	Add membership	teams.manage	userId, roleTitle	Membership
Clients	GET	/clients	List clients	clients.read	filters/page	Client page
Clients	POST	/clients	Create client account/org	clients.manage	name, contacts	Client
Projects	GET	/projects	List scoped bids/projects	projects.read	type/status/client/search	Project page
Projects	POST	/projects	Create bid/direct project	projects.create	type, client, naming, intake docs	Project
Projects	GET	/projects/:id	Read workspace	project.read scoped	-	Project aggregate
Projects	PATCH	/projects/:id	Update metadata	projects.update	patch + version	Project
Projects	POST	/projects/:id/outcome	Win/lose bid	bids.decide	WIN/LOSE, reason	Updated project
Projects	POST	/projects/:id/archive-review	Reopen archived bid review	bids.review	note	Project/request
Work Requests	GET	/work-requests	List scoped requests	work_requests.read	project/status/assignee/search	Request page
Work Requests	POST	/work-requests	Create request under parent	work_requests.create	parent, details, files	Work request
Work Requests	GET	/work-requests/:id	Read detail/actions/history	request.read scoped	-	Aggregate + availableActions
Workflow	POST	/work-requests/:id/actions/:action	Execute state transition	Action-specific	comment, assignments, file refs, expectedVersion	Updated aggregate
Info Requests	POST	/work-requests/:id/info-requests	Request information	request.info.create	target, note	Information request
Info Requests	POST	/info-requests/:id/respond	Respond to information	request.info.respond scoped	response, files	Updated request
Documents	POST	/uploads/presign	Create presigned upload	documents.upload scoped	name, size, MIME, checksum	URL/object key
Documents	POST	/documents	Register uploaded document/version	documents.create scoped	project, request, object, metadata	Document/version
Documents	GET	/projects/:id/documents	List explorer	documents.read scoped	folder/search/page	Nodes

Module	Method	Endpoint	Purpose	Authorization	Request	Response
			tree/metadata			
Documents	GET	/document-versions/:id/download	Authorized signed download	documents.download scoped	-	302/signed URL
Documents	GET	/document-versions/:id/preview	Preview status/URL	documents.read scoped	-	Preview descriptor
Documents	POST	/work-requests/:id/final-package	Select final package versions	workflow final-stage permission	versionIds	Package
Notifications	GET	/notifications	List user inbox	Authenticated	read/status/page	Notification page
Notifications	POST	/notifications/:id/read	Mark read	Recipient	-	204
Notifications	POST	/notifications/read-all	Mark all read	Authenticated	optional before	count
Audit	GET	/audit-events	Search audit history	audit.read	resource/actor/date/action	Audit page
Reports	GET	/reports/workflow-summary	Workflow KPI report	reports.read	date/division/client	Report
Reports	POST	/exports	Queue export	reports.export	type/filters/format	Job
Settings	GET	/settings	Read effective settings	settings.read	-	Settings + version
Settings	PATCH	/settings	Update settings	settings.manage	patch + expectedVersion	Settings
Health	GET	/health/live	Liveness	Internal/public minimal	-	Status
Health	GET	/health/ready	Dependency readiness	Internal	-	Status

12.1 Standard error contract

```
{
  "error": {
    "code": "WORKFLOW_TRANSITION_NOT_ALLOWED",
    "message": "This action is no longer available for the current workflow state.",
    "details": [{"field": "expectedVersion", "reason": "stale"}],
    "correlationId": "01J..."
  }
}
```

12.2 Workflow endpoint side effects

- Validate current status, actor policy, assignments, required documents, required comment, and expected aggregate version.
- Apply transition and assignments in one transaction.
- Insert workflow event and audit event.
- Create outbox records for notifications/email/preview jobs.
- Return updated status, assignments, available actions, and row version.

13. Authorization Model

Use RBAC for broad capabilities and ABAC/resource policies for the actual record. A role alone must never grant access to every project. The policy evaluates tenant/organization, client, division, assignment, workflow stage, and specific permission.

Permission	Operation	Resource policy
projects.read	Read a project/bid	Supervisor/CCR all; client own; internal actor through accessible work request or division policy.
projects.create	Create bid/direct project	System admin, Prime Consultant, CCR.
bids.decide	Win/lose accepted bid	Authorized CCR/consultant role and readiness guard.
work_requests.create	Create request	CCR/supervisor under accessible parent.
work_requests.assign_division	Assign division	CCR/supervisor.
work_requests.assign_member	Assign PM member	Assigned/origin lead or supervisor.

Permission	Operation	Resource policy
workflow.tms.assign	Assign M1/M2/M3	TMS manager or supervisor.
workflow.drawing.submit	Submit drawing	Assigned M1 or supervisor.
workflow.check.decide	M2 approval/revision	Assigned M2 or supervisor.
workflow.approval.decide	M3 approval/revision	Assigned M3 or supervisor.
workflow.client.decide	Client accept/reject/revise	Client user belonging to project client or supervisor impersonation with audit.
documents.upload	Upload document version	Actor can read request and current stage permits upload.
documents.download	Download version	Actor can read owning project/request and object classification.
settings.manage	Edit portal catalogs	System admin/Prime or explicit configuration administrator.
audit.read	Search audit events	Auditor/admin/consultant scope.

13.1 Policy evaluation example

```
allow(action, actor, workRequest) when:
  actor.isActive
  AND actor.tenantId == workRequest.tenantId
  AND actor has permission(action)
  AND one of:
      actor is supervisor
      actor.divisionId matches assigned/origin division for lead actions
      actor.userId matches assigned member/TMS assignment
      actor.clientId matches parent project client for client actions
  AND workflowEngine.can(action, workRequest.status, workRequest.assignments)
  AND required documents/comments are present
```

14. Workflow Engine Design

Move the transition map out of React components and into a pure domain package shared conceptually by API tests and type contracts. Only the API executes transitions. The frontend receives a descriptive list of allowed actions from the server.

Element	Recommended design
State	Enumerated `WorkRequestStatus` persisted in DB; statuses have display metadata but not UI-specific classes.
Event/action	Typed command such as `ASSIGN_PM_MEMBER`, `TMS_CHECK_APPROVE`, `CLIENT_REQUEST_REVISION`.
Guard	Pure predicate for current state, role/resource policy, assignment, required fields/files, and expected version.
Reducer/transition	Returns next state and domain effects; no database or email calls in pure transition function.
Side effects	Assignments, package links, registry entry, notifications, and outbox events executed transactionally by application service.
Concurrency	Optimistic lock with `version` column; transition requires `expectedVersion`; conflict returns 409.
Idempotency	Client-generated idempotency key for commands that can be retried.
History	Immutable workflow event records; display history is a projection.
Configuration	First release uses code-defined workflow; later templates may be versioned by work-request type.

```
type TransitionDefinition = {
  action: WorkflowAction;
  from: WorkRequestStatus[];
  to: WorkRequestStatus | ((ctx: Context) => WorkRequestStatus);
  requiredPermissions: Permission[];
  guard: (ctx: Context, command: Command) => GuardResult;
  effects: (ctx: Context, command: Command) => DomainEffect[];
};
```

14.1 Invalid transition behavior

- 404 when the aggregate is not visible to the actor; do not leak existence.
- 403 when actor can see the request but lacks the action permission.
- 409 when status/version changed after the page was loaded.
- 422 when required assignment, comment, or document is missing.
- Every rejected command is logged with correlation ID, but sensitive policy details are not returned to clients.

15. Document Storage Design

Concern	Production requirement
Upload	API issues short-lived presigned multipart URL after permission, MIME, extension, size, and quota checks.
Integrity	Client supplies SHA-256; worker verifies size/checksum after object arrival.
Malware	Object initially quarantined; scan job marks CLEAN/INFECTED/FAILED; only CLEAN versions are downloadable.
Metadata	Database stores original/generated names, document/revision/version fields, object key, size, MIME, checksum, uploader, timestamps.
Versioning	Create immutable `document_version`; never overwrite object. Stable document identity groups versions.
Preview	Worker creates sanitized image/PDF preview; Office formats converted in isolated worker.
Download	Policy-checked endpoint returns short-lived signed URL; log access for sensitive documents.
Retention/deletion	Soft delete metadata, object retention policy, legal hold option, admin purge job.
Backup	Versioned bucket, replication where required, DB backup with restore drills.
Naming	Generate display/path metadata server-side; object keys should be opaque UUIDs, not business filenames.

Why base64/localStorage must be removed

Base64 increases binary size, duplicates data in memory and serialized JSON, is constrained by browser quota, cannot be scanned centrally, has no cross-user consistency, and makes access control impossible. Production files belong in object storage; the database stores metadata and references only.

16. Notification Design

Capability	Recommended design
Event source	Transactional outbox entries from workflow/project/document

Capability	Recommended design
	commands.
Recipient resolution	Policy service resolves explicit assignees, role/division targets, client users, watchers, and supervisors.
In-app	Durable notifications table with resource deep link, category, priority, action-required flag, read/dismiss time.
Real-time	SSE pushes a lightweight invalidation/event; client refetches notification query.
Email	Template service with localized subject/body and secure deep link; no sensitive attachment in email.
Retries	Queue exponential backoff, max attempts, dead-letter queue, operational alert.
Deduplication	Unique event-recipient-channel key or idempotency key.
Preferences	Per-user channel/category settings; mandatory security/action notifications cannot be disabled.
Reminders/escalations	Scheduled jobs based on SLA due dates and unresolved action-required notifications.
Read status	Stored server-side and synced across devices.

16.1 Prototype notification behavior

The current notification item is produced only when `actorHasWorkAction()` is true and the work request has a latest history entry. The notification ID is normally the latest history ID. Per-actor inbox/read keys are stored in localStorage and the inbox is capped at 150 entries. This is useful as a UX prototype but not as a delivery guarantee.

17. Audit and Activity Logging

Field	Required production value
actor	User ID, effective role/permission, organization/client context, optional impersonator.
action	Stable machine action plus human summary.
resource	Type, ID, tenant, project/work-request IDs.
before/after	Redacted structured snapshots or JSON Patch for business-relevant fields.
time	Server UTC timestamp.
request context	IP, user agent, correlation/trace ID, session ID.
workflow context	Previous status, next status, expected/actual version, comment/reason.
document context	Document/version/object/checksum IDs, never raw file contents.
integrity	Append-only permissions; retention policy; optional hash chaining/export to SIEM.

Current `HistoryEntry` records are useful business activity projections, but they are embedded in mutable browser data and contain actor labels instead of immutable user IDs. Keep a friendly activity feed, but derive it from immutable workflow/audit events.

18. Validation and Business Rules

Rule area	Confirmed prototype rule	Recommended production rule
Create company	Name and abbreviation required.	Also enforce unique abbreviation/name per tenant and length/character limits.
Create division	Parent, name, abbreviation required.	Unique within organization; prevent orphan records.
Create team	Name, company, division required; duplicate name in division rejected.	Lead must belong to division; transactionally manage membership.
Create member	Name/email/company/division/role required; email format and duplicate checked.	Normalize email; invitation/account state; unique tenant email.
Create bid/project	Name/client; BID additionally bidding no., POL/POD, cargo/vessel names+codes; at least one initial doc.	Server-generated unique code, authorized client, validated catalogs, idempotency.
Create work request	Title and valid existing parent required; parent type derived from parent.	Parent state must allow new request; division/type/category must be active settings.
Upload file	At least file or note; documents explorer requires work request.	MIME/size/checksum/scan/quota and stage policy.
Assignments	UI requires selected member IDs.	All assignees active, in expected division/team, no duplicate incompatible assignment.
Workflow action	Status/role checks and some action-specific selections/comments.	Central schema per action; expectedVersion and idempotency.
Bid outcome	At least one related HML_LISTED request.	Define whether all work requests must be accepted; stakeholder confirmation required.
Final package	Latest uploaded/selected attachment IDs.	Only clean, visible, current versions; no deleted/quarantined file.
Revision code	Count matching revision uploads; BID letters / project two-digit number.	Use transactional sequence per document/revision family to prevent duplicates.

19. Error, Warning, and Empty-State Behavior

Situation	Prototype behavior	Production standard
No work request for selected project/bid	Warning toast: create a work request before opening cycle page.	Provide primary CTA and disable impossible navigation.
Invalid work-request parent	Error toast asks user to close/reopen modal.	409/422 response and automatic parent refresh.
Large local upload/persistence	Console warning or create error mentions browser storage limit.	Upload size policy and direct storage with progress/retry.
Missing required form data	Inline flow stops with toast error.	Field-level errors, summary, focus management, accessible descriptions.
Unsupported preview	Message instructs download.	Preview status, safe fallback, and conversion retry.
Older metadata file	Metadata download fallback.	Migration or explicit unavailable state.
No available workflow action	Action panel shows role/stage message.	API returns allowed actions and reason code; refresh on stale state.
No notifications/items/folder files	Dedicated empty-state text/icons.	Preserve; add role-relevant CTA.
Unexpected route error	Root generic error page, dev-only message, retry/home.	Error tracking correlation ID and safe recovery.

20. Security Requirements

Control	Requirement
Authentication	OIDC or secure first-party login; Argon2id if passwords; MFA option; client

Control	Requirement
	invitations.
Session	HttpOnly Secure SameSite cookies; short access lifetime; rotating refresh/session tokens; revoke on logout/password/role change.
Authorization	Backend RBAC+ABAC for every endpoint and file operation; deny by default.
CSRF	SameSite plus anti-CSRF token for cookie-authenticated mutations.
XSS	React escaping, CSP, sanitize rich text/preview, no unsafe HTML, safe file content disposition.
Validation	Zod/OpenAPI validation at boundary plus DB constraints.
File security	Allowlist/denylist, size limits, checksums, quarantine, malware scan, preview isolation, signed URLs.
Rate limiting	Per IP/user/client for auth, uploads, search, and high-cost exports.
Secrets	Secret manager; never commit; rotation and environment separation.
Encryption	TLS in transit; managed encryption at rest for DB/object store; field-level encryption for selected secrets.
CORS	Exact production origins; credentials only where required.
CSP	Restrictive default-src, frame-src for approved previews, object-src none, nonce/hash scripts.
Dependencies	Lockfiles, Dependabot/Renovate, npm audit/SCA, container scanning, SBOM.
Privacy/isolation	Tenant/client row policies, log redaction, retention, export/delete policy.
Admin actions	Step-up authentication or confirmation for role changes, archive, purge, and impersonation.

21. Performance and Scalability

Finding	Impact	Recommendation
780.28 KB main client chunk	Slow initial parse/execute on lower-end devices.	Route-level lazy loading, manual vendor chunks, defer rarely used dialogs/features.
Large route files/components	Hard to optimize and triggers broad rerenders.	Feature modules, smaller components, memoized selectors.
Single context value contains whole state	Any mutation can rerender all consumers.	Server cache/query subscriptions and scoped UI contexts.
Client-side filtering of all data	Degrades as projects/history grow.	Server pagination, indexed search/filter, virtualized tables when needed.
Embedded base64 files	Memory and quota explosion.	Object storage and streaming previews/downloads.
History arrays grow indefinitely	Payload and render growth.	Paginated event history and aggregate summaries.
Repeated action-policy calculations	CPU duplication and consistency risk.	Server-provided available actions plus shared pure policy package.
Document tree rebuilt from all files	Expensive with large repositories.	Folder/query API, pagination, cached projections.

21.1 Suggested service targets

Metric	Initial target
Web LCP	< 2.5 s on standard corporate connection

Metric	Initial target
API p95 read	< 400 ms excluding large downloads
API p95 workflow command	< 800 ms before async notifications
Notification delivery	In-app event < 5 s; email queue start < 60 s
Document upload	Direct multipart with resumability for configured maximum size
Availability	99.9% initial target, subject to business confirmation
Recovery	RPO <= 15 min; RTO <= 4 h initial target, stakeholder-approved

22. Testing Strategy

Layer	Tools	Priority coverage
Unit/domain	Vitest	Naming normalization, revision sequence, permission predicates, every transition guard/reducer.
Frontend component	Vitest + Testing Library	Forms, tables, dialogs, empty/error states, keyboard/a11y.
API integration	Vitest/Jest + test DB	Auth, policy, workflow transactions, outbox, validation, pagination.
Database	Prisma migrations + integration tests	Constraints, indexes, concurrency, rollback, seed.
Storage/jobs	MinIO/test queue	Presign, finalize, scan states, preview, retry/DLQ.
End-to-end	Playwright	Full CCR-to-client path, revisions, role switching via real accounts, document lifecycle.
Security	SAST/SCA, OWASP ZAP, policy tests	IDOR, privilege escalation, upload bypass, session/CSRF/XSS.
Performance	k6/Artillery + Lighthouse	List/search, workflow burst, notification fanout, upload/download, bundle budgets.

22.1 High-priority workflow test cases

11. CCR creates a bid, attaches intake document, creates a work request, and assigns ECM.
12. Division lead requests Marketing info; Marketing escalates to Client; Client responds; package returns to PM.
13. PM member requests clarification, receives response, submits, is returned for rework, and resubmits.
14. TMS manager assigns M1/M2/M3; unauthorized users cannot act on another assignment.
15. M2 and M3 rejection paths return to M1 and preserve version/revision history.
16. TMS engineering lead requests PM revision and later submits a specific final package.
17. CCR requests engineering revision or submits final to client.
18. Client requests revision; CCR routes to TMS; revised package is delivered and accepted.
19. Client acceptance is idempotent and creates only one registry entry.
20. Concurrent actors attempt the same transition; one succeeds and one receives 409.
21. Infected upload is never selectable/downloadable.
22. Client A cannot infer or access Client B projects, requests, files, or notifications.

23. Deployment and DevOps

Area	Recommended implementation
Local development	Docker Compose for PostgreSQL, Redis, MinIO, mail catcher; pnpm/Nx commands; seeded accounts.
Environment variables	Validated at startup; separate web-public and server-secret schemas;

Area	Recommended implementation
	`.env.example` without secrets.
CI	Install with frozen lockfile, format check, lint, type check, unit/integration tests, build, SCA, secret scan.
Migrations	Prisma migration artifact reviewed in PR; deploy migration before compatible app release; rollback/forward-fix plan.
Images	Multi-stage non-root containers, pinned base images, health checks, SBOM/signing.
Staging	Production-like dependencies, anonymized/seed data, approval gate for release.
Production	Managed container platform or Kubernetes only if operational maturity justifies it; CDN for web/previews.
Observability	Structured JSON logs, trace IDs, metrics dashboards, error tracking, audit search.
Backups	Automated DB/object policies, cross-region option, scheduled restore tests.
Rollback	Blue/green or rolling app rollback; backward-compatible DB changes; feature flags for risky modules.

```
CI quality gates (recommended):
1. pnpm install --frozen-lockfile
2. nx format:check
3. nx affected -t lint typecheck test build
4. API integration tests with ephemeral PostgreSQL/Redis/MinIO
5. Playwright smoke suite
6. dependency, secret, container, and IaC scans
7. publish immutable artifacts only after all gates pass
```

24. Prototype Gap Analysis

Area	Current	Limitation	Production need	Priority	Complexity	Solution
Authentication	Actor dropdown	No identity/session	Real employee/client auth	Critical	High	OIDC/secure session
Authorization	Frontend visibility checks	Bypassable	API/storage enforcement	Critical	High	RBAC+ABAC policy layer
Database	localStorage JSON	No concurrency/durability	Transactional source of truth	Critical	High	PostgreSQL/Prisma
File storage	Base64 data URLs	Quota/security/ performance	Object storage pipeline	Critical	High	S3/MinIO + scan
Workflow engine	Callbacks and UI switches	Duplicate/inconsistent logic	Central state machine	Critical	High	Domain package/API service
Notifications	Derived local inbox	No delivery guarantee	Durable multichannel service	High	Medium	Outbox + queue + SSE/email
Audit	Mutable history labels	Not forensic	Immutable audit events	High	Medium	Audit/outbox tables
Concurrency	None	Lost/duplicate actions	Optimistic locking/idempotency	Critical	Medium	Version columns + keys
Validation	Manual toasts	Inconsistent/client-only	Shared schemas + DB constraints	High	Medium	Zod/OpenAPI/Prisma
Client isolation	Array filter by clientId	Not secure	Tenant/resource policies	Critical	High	ABAC and query scopes
Email	Simulated credentials	No delivery/retry	Email provider + templates	High	Medium	Worker queue
Reporting	Dashboard counts only	No exports/analytics	Operational reports	Medium	Medium	Report queries/jobs
Testing	No tests found	Regression risk	Automated pyramid	Critical	High	Vitest/Playwright/ integration
Monitoring	Console only	Poor incident response	Logs/metrics/traces/errors	High	Medium	OpenTelemetry stack
Performance	780 KB main chunk	Slow growth path	Budgets/lazy loading	Medium	Medium	Route splitting

Area	Current	Limitation	Production need	Priority	Complexity	Solution
Deployment	Prototype Cloudflare config	No full stack pipeline	Staged, repeatable release	High	High	Containers + CI/CD

25. Technical Debt and Code Quality Review

Severity	Issue	Evidence	Impact	Fix
Critical	No server security boundary	All identity/permissions/state are client-controlled.	Data breach and unauthorized actions.	Build API auth/policy before production data.
Critical	Base64/localStorage file persistence	`PortalDataProvider` serializes file data URLs.	Quota failure, leakage, unusable scale.	Object storage and metadata DB.
High	God context/provider	3,087-line `portal-data.tsx` combines models, seed, naming, persistence, workflow.	Tight coupling and fragile changes.	Domain packages and API services.
High	Very large routes	Work Requests 5,561 lines; Dashboard 4,108; Projects 1,586; Documents 1,199.	Low maintainability/testability.	Feature decomposition.
High	Duplicated workflow policy	Dashboard and Work Requests each calculate actions/targets.	Drift and inconsistent permissions.	One server policy + shared descriptors.
High	Unreachable function path	Marketing-to-PM client revision function exists; UI permission returns false.	Ambiguous business behavior.	Stakeholder decision and remove/enable.
High	No tests	No test files/scripts in source package.	Workflow regressions.	Domain tests before refactor.
Medium	JSON deep clone mutations	`clone()` uses stringify/parse.	Performance and type limitations.	Immutable DB transactions; targeted client updates.
Medium	Random short IDs and count-based codes	Math.random and array counts.	Collision/race risk.	UUID/ULID and DB sequences.
Medium	Manual form validation	Many imperative toast checks.	Inconsistent UX and schema drift.	RHF/Zod contracts.
Medium	Legacy states/actions coexist	RETURNED_TO_DIVISION and legacy listing path remain.	Confusing state graph.	Versioned workflow migration.
Medium	Lint failure	741 findings: 698 errors, 43 warnings.	Quality gate cannot pass.	Format then resolve hooks/Fast Refresh.
Medium	Bundle size	780.28 KB client main chunk.	Slower load/parse.	Lazy routes and chunk budgets.
Low	Hardcoded seed/business options	Organizations, clients, users, ports, codes in source.	Configuration changes require code/local state.	Admin-managed tables with audit/versioning.
Low	Role title regex	Lead/manager detection uses text regex.	Brittle permission behavior.	Explicit role assignments.

26. Recommended Monorepo Structure

```
project-portal/
├── apps/
│   ├── web/                # React/Vite portal
│   │   └── src/
│   │       ├── app/        # providers, router, layouts, config
│   │       ├── features/   # auth, dashboard, bids, projects, work-requests,
│   │       │               # workflow, documents, notifications, org, settings
│   │       └── shared/     # app-level composition helpers
│   ├── api/                # NestJS modular monolith
│   │   └── src/modules/    # auth, users, org, clients, projects, workflow,
│   │                       # documents, notifications, audit, reports, settings
│   ├── worker/             # queues: email, scan, preview, reminders, exports
│   └── storybook/          # design-system catalog and visual tests
├── packages/
│   ├── ui/                 # reusable presentation components only
│   ├── design-tokens/      # colors, spacing, typography, breakpoints
│   ├── contracts/          # Zod DTOs, enums, API error/page contracts
│   ├── api-client/         # generated/typed client and query keys
│   ├── permissions/        # permission names and pure policy helpers
│   ├── workflow-engine/    # states, actions, guards, transitions, effects
│   ├── file-naming/        # naming/folder/version/revision rules
```

```
│   ├── database/            # Prisma schema/client/repositories
│   ├── storage/             # S3/MinIO provider interfaces
│   ├── notifications/       # event and template contracts
│   ├── logger/              # structured logging/trace helpers
│   ├── testing/             # fixtures, test builders, MSW helpers
│   ├── eslint-config/
│   └── tsconfig/
├── infra/
│   ├── docker/
│   ├── terraform/           # or chosen IaC
│   ├── kubernetes/          # only if selected deployment requires it
│   └── observability/
├── docs/
│   ├── architecture/
│   ├── workflows/
│   ├── api/
│   ├── database/
│   ├── permissions/
│   └── decisions/           # ADRs
├── tools/                   # Nx generators, scripts, migrations/imports
├── .github/workflows/
├── nx.json
├── pnpm-workspace.yaml
├── package.json
└── tsconfig.base.json
```

26.1 Package boundary rules

Package/app	May import	Must not import
apps/web	ui, contracts, api-client, permission display helpers	database, storage internals, API repositories
apps/api	contracts, permissions, workflow-engine, database, storage, notifications	React/UI packages
apps/worker	contracts, database, storage, notifications, logger	Web feature code
packages/ui	design-tokens and generic utility types	Business domain features or API/database
packages/workflow-engine	contracts and pure utilities	NestJS, React, Prisma, Redis, file system
packages/contracts	Zod and shared scalar helpers	Database entities or framework controllers

26.2 Frontend feature template

```
features/work-requests/
├── api/          # queries, mutations, query keys
├── components/   # table, card, status, action presentation
├── dialogs/      # create, assign, info, revision
├── hooks/        # orchestration hooks
├── pages/        # route-level screens
├── schemas/      # form schemas composed from contracts
├── types/
├── utils/        # presentation-only formatters
└── index.ts      # intentional public API
```

27. Development Roadmap

Phase	Objectives	Deliverables/acceptance	Primary risk
1. Discovery	Confirm workflow/statuses, roles, all document rules, archive/client-pending behavior.	Approved requirements, transition matrix, glossary, open-question closure.	Stakeholder availability.
2. Monorepo setup	Nx/pnpm, apps/packages, CI, local Docker dependencies.	Green lint/type/test/build pipeline.	Tooling discipline.

Phase	Objectives	Deliverables/acceptance	Primary risk
3. Design system	Tokens, layout, forms, tables, dialogs, accessibility, Storybook.	Reusable UI and prototype parity components.	Uncontrolled one-off UI.
4. Auth/authorization	User/client accounts, sessions, invitations, RBAC/ABAC.	Secure login and policy test suite.	Identity provider choice.
5. Organization/users	Companies, divisions, teams, members, clients.	CRUD, validation, audit, seed/import.	Data ownership rules.
6. Bids/projects	Create/read/filter, naming data, intake documents, archive.	Production project workspace.	Numbering migration.
7. Work requests	Create/list/detail, assignment, comments/info requests.	Request aggregate and UI.	Scope complexity.
8. Workflow engine	Implement/test transitions, policies, concurrency, events.	Server authoritative state machine.	Unresolved legacy paths.
9. Documents	Presigned upload, scan, versioning, explorer, preview, download.	Secure document lifecycle.	File conversion infrastructure.
10. Notifications	Outbox, in-app, SSE, email, retries/preferences.	Durable action queues.	Email/domain setup.
11. Client portal	Client-scoped projects, info, delivery, decisions, revisions.	Secure external experience.	Client identity/terms.
12. Audit/reports	Audit search, activity projection, KPIs/exports.	Operational oversight.	Retention/report definitions.
13. Hardening	E2E, security, performance, accessibility, disaster recovery.	Release candidate and test evidence.	Defect backlog.
14. Migration/release	Import validated prototype/reference data, training, staged rollout.	Production launch and rollback plan.	Data quality/change management.

27.1 Recommended implementation order

Do not begin by recreating every prototype screen. First lock the workflow transition matrix, authorization scopes, and document identity/version rules. Then implement the backend aggregate and automated transition tests before connecting the full UI. This prevents the new React application from becoming another source of business logic.

28. Acceptance Criteria

Module	Given / When / Then
Authentication	Given an inactive or revoked user, when a session is used, then protected endpoints return unauthorized and no resource data.
Client isolation	Given two clients, when Client A requests Client B project/file ID, then the API returns not found/forbidden and logs the attempt.
Create bid	Given valid CCR permissions and naming data, when a bid is submitted with a clean intake file, then a unique BID is created and audited.
Create work request	Given an accessible valid parent, when CCR submits required details, then a request is created in the initial division-review state.
Invalid parent	Given a deleted/inaccessible parent, when creation is attempted, then no request is created and a field/business error is returned.
Assignment	Given a PM lead, when assigning an inactive/out-of-division member, then the command is rejected.
Permission failure	Given a non-assigned member, when attempting another member action, then the command is denied even if the button was manually invoked.
Workflow concurrency	Given two actors on the same version, when both transition, then only the first succeeds and the second receives 409 with refreshed state.
M2 revision	Given CHECKING_REVIEW, when M2 requests revision with comment, then state returns to drawing, comment/event is stored, and M1 is notified.
M3 approval	Given APPROVAL_REVIEW, when M3 approves, then state moves to engineering lead review and manager receives action notification.

Module	Given / When / Then
PM revision	Given engineering lead review, when PM revision is requested, then PM lead becomes current handler and selected files remain traceable.
Final package	Given clean visible versions, when the engineering lead submits final package, then exact version IDs are immutably associated.
Client revision	Given final delivery, when client uploads revision files and requests revision, then files are linked to that revision cycle and CCR is notified.
Client acceptance	Given final delivery, when client accepts twice using same idempotency key, then one acceptance/event/registry entry exists.
Bid conversion	Given an accepted bid ready for outcome, when marked won, then it becomes active project without losing document/work-request history.
Upload quarantine	Given a new file, when scan is pending or infected, then it cannot be previewed, selected, or downloaded.
Versioning	Given the same document identity, when a new file is finalized, then a new immutable version number is assigned transactionally.
Notifications	Given a committed action event, when notification processing retries, then recipients receive no duplicates and delivery status is recorded.
Audit	Given any successful or denied sensitive mutation, when reviewed by an auditor, then actor, resource, action, time, and correlation context are available.
Accessibility	Given keyboard-only navigation, when using forms/dialogs/tables, then focus order, labels, errors, and modal trapping meet WCAG 2.1 AA targets.

29. Glossary

Term	Definition
CCR	Marketing/coordinator division responsible for intake, routing, client communication, and final delivery/listing.
ECM	Engineering division represented by division lead/member roles.
PMO	Operations/project-management division represented by the same lead/member role types.
TMS	Technical Management Service engineering workflow.
TMS-M1	Drawing member.
TMS-M2	Checking member.
TMS-M3	Approval member.
Bid	Proposal-stage parent workspace.
Project	Active execution workspace, direct or converted from a won bid.
Work Request	Unit of work under one bid/project.
Primary file	Initial project/bid intake document.
Workflow file	Document uploaded during work-request collaboration or actions.
Document version	Sequential version of the same stable document identity.
Revision code	Structured code on a revision upload: letters for bids, two-digit numbers for projects.
Workflow revision	A business return/rework cycle represented by status and events.
Client revision	Revision requested by client after final submission.
Internal TMS revision	Engineering/M2/M3 return to M1.
Final package	Selected exact document versions delivered through final stages.

Term	Definition
HML listed	Accepted/final logical registry state.
Client pending	Prototype UI meaning of a bid set to DRAFT after client acceptance but before win/loss assignment.
Information request	Targeted question to a role/member/client that can be stacked and responded to.
Outbox	Database pattern that records events in the same transaction as business changes for reliable async delivery.
RBAC	Role-based access control.
ABAC	Attribute/resource-based access control.
Presigned URL	Short-lived object-storage upload/download URL.
Optimistic lock	Version check preventing stale concurrent updates.
Idempotency key	Client-provided key that prevents duplicate command effects during retry.

30. Open Questions

23. Must every work request under a bid be accepted before the bid can be marked won/lost, or is one HML-listed request sufficient as the prototype currently checks?
24. Is the post-client-acceptance `DRAFT` bid status officially named Client Pending, and who is allowed to assign it as a project?
25. Should Marketing be able to route a client revision to PM directly? The function exists, but the current UI disables the action.
26. Are the legacy RETURNED_TO_DIVISION -> member -> manager -> CCR -> registry stages still required, or should they be removed?
27. Can M1, M2, and M3 be the same user? Are substitutes/delegates allowed?
28. Can multiple TMS assignment chains exist per revision cycle, and must prior assignments remain immutable?
29. Which actions require mandatory comments and/or mandatory document versions?
30. What is the authoritative numbering scheme across years/tenants for bids, projects, work requests, revisions, and document versions?
31. Are document naming options global, tenant-specific, or client/project-specific?
32. Which file types and maximum sizes are allowed, and what retention/legal-hold requirements apply?
33. Should clients upload directly into work requests, or only into a controlled client exchange area?
34. Do final acceptance/rejection actions require formal acknowledgment, signature, or delivery receipt?
35. What are the notification SLA, reminder cadence, escalation recipients, and channel preferences?
36. What reports and exports are mandatory for operations, management, and audit?
37. Is multi-tenancy required beyond client scoping, and are HML/TMS separate legal tenants or organizations inside one tenant?
38. Which identity provider and deployment environment are approved?
39. What prototype data, if any, must be migrated to production?

Appendix A. Confirmed Status Catalog

Status code	Display label
CREATED	Created
DIVISION_NOTIFIED	PM Lead Initial Review
INFO_REQUESTED_FROM_MARKETING	PM Requested Info from Marketing
INFO_REQUESTED_FROM_CLIENT	Marketing Requested Info from Client
CLIENT_INFO_PROVIDED	Client Info Provided to Marketing
MARKETING_INFO_PROVIDED	Marketing Info Returned to PM
LEADER_ASSIGNED	PM Lead Assigned
MEMBER_REVIEW	PM Member Work
MEMBER_INFO_REQUESTED	PM Member Requested Info
PM_LEAD_RESPONDED	PM Lead Responded to Member
PM_MEMBER_SUBMITTED	PM Member Submitted to Lead
PM_REWORK_REQUESTED	PM Lead Requested Rework
FORWARDED_TO_TMS	Forwarded to TMS Engineering Lead
ENGINEERING_REVISION_REQUESTED	Engineering Requested PM Revision
TMS_MEMBER_INFO_REQUESTED	TMS Member Requested Lead Info
TMS_ASSIGNED	TMS Engineering Chain Assigned
DRAWING_IN_PROGRESS	TMS-M1 Drawing
CHECKING_REVIEW	TMS-M2 Checking
APPROVAL_REVIEW	TMS-M3 Approval
ENGINEERING_LEAD_REVIEW	Engineering Lead Final Review
FINAL_SUBMITTED_TO_MARKETING	Final Submitted to Marketing
FINAL_SUBMITTED_TO_CLIENT	Final Submitted to Client
CLIENT_REVISION_REQUESTED	Client Requested Revision
RETURNED_TO_DIVISION	Returned to PM Member
DIVISION_MEMBER_APPROVED	PM Member Approved
DIVISION_MANAGER_APPROVED	PM Lead Approved
FORWARDED_TO_CCR	Forwarded to CCR
HML_LISTED	Client Accepted / HML Listed
REJECTED	Rejected

Appendix B. Generic Backward Transition Map

From status	Backward status
LEADER_ASSIGNED	DIVISION_NOTIFIED
MEMBER_REVIEW	LEADER_ASSIGNED
INFO_REQUESTED_FROM_MARKETING	DIVISION_NOTIFIED
INFO_REQUESTED_FROM_CLIENT	INFO_REQUESTED_FROM_MARKETING
CLIENT_INFO_PROVIDED	INFO_REQUESTED_FROM_CLIENT

From status	Backward status
MARKETING_INFO_PROVIDED	INFO_REQUESTED_FROM_MARKETING
MEMBER_INFO_REQUESTED	MEMBER_REVIEW
PM_LEAD_RESPONDED	MEMBER_INFO_REQUESTED
PM_MEMBER_SUBMITTED	MEMBER_REVIEW
PM_REWORK_REQUESTED	PM_MEMBER_SUBMITTED
FORWARDED_TO_TMS	PM_MEMBER_SUBMITTED
TMS_ASSIGNED	FORWARDED_TO_TMS
DRAWING_IN_PROGRESS	TMS_ASSIGNED
CHECKING_REVIEW	DRAWING_IN_PROGRESS
APPROVAL_REVIEW	CHECKING_REVIEW
ENGINEERING_REVISION_REQUESTED	FORWARDED_TO_TMS
ENGINEERING_LEAD_REVIEW	APPROVAL_REVIEW
FINAL_SUBMITTED_TO_MARKETING	ENGINEERING_LEAD_REVIEW
FINAL_SUBMITTED_TO_CLIENT	FINAL_SUBMITTED_TO_MARKETING
CLIENT_REVISION_REQUESTED	FINAL_SUBMITTED_TO_CLIENT
RETURNED_TO_DIVISION	APPROVAL_REVIEW
DIVISION_MEMBER_APPROVED	RETURNED_TO_DIVISION
DIVISION_MANAGER_APPROVED	DIVISION_MEMBER_APPROVED
FORWARDED_TO_CCR	DIVISION_MANAGER_APPROVED

Evidence: `sendBackward()` in `src/lib/portal-data.tsx` lines 2603-2745. Production should not expose a generic backward action unless each transition is explicitly approved and audited; use named business actions instead.

Appendix C. Mermaid Diagrams

C.1 Current architecture

```
flowchart LR
  Browser[React browser app] --> Router[TanStack Router]
  Router --> Routes[Dashboard and feature routes]
  Routes --> Context[PortalDataProvider]
  Context --> LocalStorage[(localStorage state and data URLs)]
  Context --> Notifications[Derived notification inbox]
  Routes --> UI[Radix/ShadCN UI primitives]
```

C.2 Recommended production architecture

```
flowchart LR
  Web[React/Vite Web] --> API[NestJS API]
  API --> DB[(PostgreSQL/Prisma)]
  API --> Redis[(Redis)]
  API --> Storage[(S3/MinIO)]
  API --> Outbox[(Transactional Outbox)]
  Outbox --> Worker[Background Worker]
  Worker --> Email[Email Provider]
  Worker --> Storage
  API --> SSE[SSE Gateway]
  SSE --> Web
```

C.3 Core workflow

```
stateDiagram-v2
  [*] --> DIVISION_NOTIFIED
  DIVISION_NOTIFIED --> MEMBER_REVIEW: assign member
  MEMBER_REVIEW --> PM_MEMBER_SUBMITTED: submit
  PM_MEMBER_SUBMITTED --> PM_REWORK_REQUESTED: request rework
  PM_REWORK_REQUESTED --> PM_MEMBER_SUBMITTED: resubmit
  PM_MEMBER_SUBMITTED --> FORWARDED_TO_TMS: approve/forward
  FORWARDED_TO_TMS --> TMS_ASSIGNED: assign M1/M2/M3
  TMS_ASSIGNED --> CHECKING_REVIEW: M1 submit
  CHECKING_REVIEW --> DRAWING_IN_PROGRESS: M2 revision
  DRAWING_IN_PROGRESS --> CHECKING_REVIEW: M1 resubmit
  CHECKING_REVIEW --> APPROVAL_REVIEW: M2 approve
  APPROVAL_REVIEW --> DRAWING_IN_PROGRESS: M3 revision
  APPROVAL_REVIEW --> ENGINEERING_LEAD_REVIEW: M3 approve
  ENGINEERING_LEAD_REVIEW --> FINAL_SUBMITTED_TO_MARKETING: release
  FINAL_SUBMITTED_TO_MARKETING --> FINAL_SUBMITTED_TO_CLIENT: deliver
  FINAL_SUBMITTED_TO_CLIENT --> CLIENT_REVISION_REQUESTED: revise
  CLIENT_REVISION_REQUESTED --> ENGINEERING_LEAD_REVIEW: route to TMS
  FINAL_SUBMITTED_TO_CLIENT --> HML_LISTED: accept
  FINAL_SUBMITTED_TO_CLIENT --> REJECTED: reject
```

Appendix D. Source Inventory

File	Category
.gitignore	Configuration/build
.prettierignore	Configuration/build
.prettierrc	Configuration/build
.wrangler/deploy/config.json	Configuration/build
IMPLEMENTATION_NOTE.md	Documentation
README.md	Documentation
bunfig.toml	Configuration/build
components.json	Configuration/build
eslint.config.js	Configuration/build
package-lock.json	Configuration/build
package.json	Configuration/build
src/components/AppSidebar.tsx	Shared component
src/components/PageHeader.tsx	Shared component
src/components/StatCard.tsx	Shared component
src/components/StatusBadge.tsx	Shared component
src/components/WorkflowTimeline.tsx	Shared component
src/components/ui/accordion.tsx	UI primitive
src/components/ui/alert-dialog.tsx	UI primitive
src/components/ui/alert.tsx	UI primitive
src/components/ui/aspect-ratio.tsx	UI primitive
src/components/ui/avatar.tsx	UI primitive
src/components/ui/badge.tsx	UI primitive
src/components/ui/breadcrumb.tsx	UI primitive
src/components/ui/button.tsx	UI primitive
src/components/ui/calendar.tsx	UI primitive

File	Category
src/components/ui/card.tsx	UI primitive
src/components/ui/carousel.tsx	UI primitive
src/components/ui/chart.tsx	UI primitive
src/components/ui/checkbox.tsx	UI primitive
src/components/ui/collapsible.tsx	UI primitive
src/components/ui/command.tsx	UI primitive
src/components/ui/context-menu.tsx	UI primitive
src/components/ui/dialog.tsx	UI primitive
src/components/ui/drawer.tsx	UI primitive
src/components/ui/dropdown-menu.tsx	UI primitive
src/components/ui/form.tsx	UI primitive
src/components/ui/hover-card.tsx	UI primitive
src/components/ui/input-otp.tsx	UI primitive
src/components/ui/input.tsx	UI primitive
src/components/ui/label.tsx	UI primitive
src/components/ui/menubar.tsx	UI primitive
src/components/ui/navigation-menu.tsx	UI primitive
src/components/ui/pagination.tsx	UI primitive
src/components/ui/popover.tsx	UI primitive
src/components/ui/progress.tsx	UI primitive
src/components/ui/radio-group.tsx	UI primitive
src/components/ui/resizable.tsx	UI primitive
src/components/ui/scroll-area.tsx	UI primitive
src/components/ui/select.tsx	UI primitive
src/components/ui/separator.tsx	UI primitive
src/components/ui/sheet.tsx	UI primitive
src/components/ui/sidebar.tsx	UI primitive
src/components/ui/skeleton.tsx	UI primitive
src/components/ui/slider.tsx	UI primitive
src/components/ui/sonner.tsx	UI primitive
src/components/ui/switch.tsx	UI primitive
src/components/ui/table.tsx	UI primitive
src/components/ui/tabs.tsx	UI primitive
src/components/ui/textarea.tsx	UI primitive
src/components/ui/toggle-group.tsx	UI primitive
src/components/ui/toggle.tsx	UI primitive
src/components/ui/tooltip.tsx	UI primitive
src/hooks/use-mobile.tsx	Application source
src/lib/portal-data.tsx	Library/business state
src/lib/utils.ts	Library/business state
src/routeTree.gen.ts	Application source
src/router.tsx	Application source

File	Category
src/routes/__root.tsx	Feature route
src/routes/companies.tsx	Feature route
src/routes/documents.tsx	Feature route
src/routes/index.tsx	Feature route
src/routes/notifications.tsx	Feature route
src/routes/projects.tsx	Feature route
src/routes/settings.tsx	Feature route
src/routes/teams.tsx	Feature route
src/routes/work-requests.tsx	Feature route
src/styles.css	Application source
tsconfig.json	Configuration/build
vite.config.ts	Configuration/build
wrangler.jsonc	Configuration/build

Appendix E. Primary Code Evidence Index

Topic	Evidence
Roles/entities/statuses	`src/lib/portal-data.tsx` lines 3-63, 268-456
Context API/actions	`src/lib/portal-data.tsx` lines 458-565
Naming/version/revision	`src/lib/portal-data.tsx` lines 602-940
Seed organizations/actors/projects/requests	`src/lib/portal-data.tsx` lines 942-1431
localStorage persistence	`src/lib/portal-data.tsx` lines 1433-1460
Project/work-request creation and upload	`src/lib/portal-data.tsx` lines 1541-1774
Information requests	`src/lib/portal-data.tsx` lines 1816-1888
Core PM/TMS/client transitions	`src/lib/portal-data.tsx` lines 1890-2601
Backward transitions and bid lifecycle	`src/lib/portal-data.tsx` lines 2603-2872
Status labels and simple status	`src/lib/portal-data.tsx` lines 2998-3071
Sidebar data scope	`src/components/AppSidebar.tsx` lines 116-205
Work action policy	`src/routes/work-requests.tsx` lines 1115-1395
Notifications	`src/routes/notifications.tsx` lines 68-519
Document explorer	`src/routes/documents.tsx` lines 69-488 and 600-1199
Project page business validations/actions	`src/routes/projects.tsx` lines 319-578 and 686-877
Teams validation	`src/routes/teams.tsx` lines 280-360
Settings groups/actions	`src/routes/settings.tsx` lines 70-447

Appendix F. Final Quality-Control Checklist

☑️ Every non-generated functional source route/provider/configuration file was reviewed; all 80 source/config/document files are inventoried.

☑️ All routes, actor roles, workflow statuses, principal actions, data types, storage mechanisms, notifications, revision paths, and dialogs are documented.

☑️ Prototype behavior and production recommendations are separated.

☑ No backend/email/database/cloud functionality is described as currently implemented.

☑ Uncertain or conflicting business behavior is listed under Open Questions.

☑ The production design includes authentication, authorization, database, storage, workflow, notifications, audit, security, testing, DevOps, and migration concerns.

Implementation use

Treat this document as the production discovery baseline. Before coding the workflow engine, stakeholders must approve the transition catalog and answer the open questions. After approval, freeze a versioned workflow specification and derive API contracts, automated tests, and migration rules from it.